

Chapter 12

網頁互動實作 - DOM 操 作與事件

Horazon

互動媒體設計

什麼是 DOM？

Document Object Model

- 瀏覽器把 HTML 程式碼讀進去後，會自作主張把它轉成一個樹狀結構的物件，這個物件就叫 DOM。
- JS 透過 DOM 來控制 HTML。
- 簡單來說：**DOM 就是 JS 和 HTML 溝通的橋樑。**

`document` 是這個大樹的根節點 (Root)。

抓取元素

要控制誰，先把它抓出來。

1. 透過 ID (最快)

```
const title = document.getElementById("title");
```

2. 透過 Selector (最強)

```
// 抓第一個符合的 (像 CSS 選擇器一樣)  
const box = document.querySelector(".box");  
  
// 抓所有符合的 (回傳陣列)  
const allItems = document.querySelectorAll("li");
```

修改內容

抓到元素後，我們可以改它的內容。

```
const title = document.getElementById("title");

// 1. 修改文字 (innerText)
title.innerText = "你好，JS！";

// 2. 修改 HTML (innerHTML)
title.innerHTML = "<em>斜體字</em>"; // 會解析 HTML 標籤
```

注意：`innerHTML` 有資安風險 (XSS)，如果是使用者輸入的內容，請用 `innerText`。

修改樣式

JS 也可以直接改 CSS。

```
const box = document.querySelector(".box");

// 1. 直接改 style 屬性 (駝峰式命名)
box.style.backgroundColor = "red"; // background-color -> backgroundColor
box.style.fontSize = "24px";
box.style.display = "none"; // 隱藏元素

// 2. 透過 Class 切換 (推薦!)
box.classList.add("active"); // 加上 .active
box.classList.remove("active"); // 移除 .active
box.classList.toggle("active"); // 切換 (有就刪，沒有就加)
```

事件監聽

網頁要互動，就要監聽使用者的動作 (事件)。

```
const btn = document.getElementById("myBtn");

// 當按鈕被點擊 (click) 時，執行函式
btn.addEventListener("click", function() {
    alert("你按到我了！");
});
```

常用事件：

- `click`：滑鼠點擊。
- `mouseenter` / `mouseleave`：滑鼠移入/移出。
- `keydown`：鍵盤按下。
- `submit`：表單送出。

• `onload`：網頁加載。

實作練習：計數器

做一個有 `+` 和 `-` 按鈕的計數器。

HTML:

```
<h1 id="count">0</h1>  
<button id="btn-add">+1</button>  
<button id="btn-sub">-1</button>
```

JavaScript:

1. 抓取三個元素。
2. 宣告一個變數 `num = 0`。
3. 監聽 `+` 按鈕：`num` 加 1，更新 `h1`。
4. 監聽 `-` 按鈕：`num` 減 1，更新 `h1`。

實作練習：背景變色器

做一個按鈕，按下去隨機改變網頁背景顏色。

```
const btn = document.getElementById("bg-btn");

btn.addEventListener("click", function() {
  // 產生隨機顏色
  const r = Math.floor(Math.random() * 256);
  const g = Math.floor(Math.random() * 256);
  const b = Math.floor(Math.random() * 256);

  // 套用到 body 背景
  document.body.style.backgroundColor = `rgb(${r}, ${g}, ${b})`;
});
```

`Math.random()`：產生 0 ~ 1 之間的小數。

`Math.floor()`：無條件捨去取整數。

進階應用：抓取網路資料 (fetch API)

只改自己的網頁還不夠，我們可以去「抓」別人的資料！

```
const btn = document.getElementById("dog-btn");
const img = document.getElementById("dog-img");

btn.addEventListener("click", function() {
  // 呼叫狗狗圖片 API
  fetch("https://dog.ceo/api/breeds/image/random")
    .then(res => res.json())
    .then(data => {
      img.src = data.message; // 把圖片網址換掉
    });
});
```

恭喜！你剛完成了第一次「打通網路」的體驗。

微互動：AOS 滾動動畫

想讓你的網頁像蘋果官網一樣有高級感？

不需要自己寫厚厚的 JS，只要套用 AOS (Animate On Scroll) 套件即可。

1. 在 HTML `<head>` 引入 AOS 的 CSS 與 JS 檔案庫。
2. 在你想加入動畫的元素加上 `data-aos="fade-up"`。

```
<div data-aos="fade-up" data-aos-duration="1000">  
  我會從下面飛出來！  
</div>
```

3. 在 JS 寫一行啟動碼：`AOS.init();`

這些「微小而精密」的互動，會大幅提升你最終實作的質感！

總結：DOM 操作三部曲

任何網頁互動，基本都脫離不了這三個步驟：

1. **選取**：用 `querySelector` 找到你要控制的 HTML 標籤。
2. **監聽**：用 `addEventListener` 監聽使用者的動作。
3. **反應**：在函式裡面，修改 DOM 的內容或樣式。

補充：取得輸入框的值 (Input Value)

除了點擊按鈕，我們也很常需要讀取使用者在表單填寫的資料。

```
<input type="text" id="myInput" placeholder="請輸入名字">  
<button id="sendBtn">送出</button>
```

```
const input = document.getElementById("myInput");  
const btn = document.getElementById("sendBtn");  
  
btn.addEventListener("click", function() {  
    // 使用 .value 來取得輸入框的內容  
    const userName = input.value;  
    alert("你好，" + userName + "！");  
});
```

補充：動態建立 HTML 元素

有時候我們不只要「修改」現有的元素，還需要「無中生有」創造新元素。

```
// 1. 創造一個新的 <li> 標籤
const newLi = document.createElement("li");

// 2. 設定它的內容
newLi.innerText = "這是我用 JS 動態產生的一句話！";

// 3. 把他塞進現有的 <ul> 裡面 (假設 id="myList")
const list = document.getElementById("myList");
list.appendChild(newLi);
```

這就是所謂的「資料驅動畫面」的基礎！

很多購物車、留言板都是這樣做出來的。

補充：表單事件與阻擋預設行為

當我們送出 `<form>` 表單時，網頁預設會「重新整理」。
如果我們想要自己用 JS 處理資料，就需要阻擋這個預設行為。

```
<form id="myForm">
  <input type="text" placeholder="輸入內容">
  <button type="submit">送出表單</button>
</form>
```

```
const form = document.getElementById("myForm");

form.addEventListener("submit", function(e) {
  e.preventDefault(); // 阻擋表單預設的重新整理
  alert("表單已經被 JS 接管囉！");
});
```

補充：計時器 (Timer)

如果我們想要延遲一段時間再執行程式，或者每隔一段時間重複執行：

1. `setTimeout` (延遲一次)

```
setTimeout(function() {  
    console.log("3 秒鐘到了!");  
}, 3000); // 3000 毫秒 = 3 秒
```

2. `setInterval` (不斷重複)

```
let count = 0;  
setInterval(function() {  
    count++;  
    console.log("經過了 " + count + " 秒");  
}, 1000); // 每 1000 毫秒執行一次
```

總結：前端三劍客的組合

到這裡，你已經掌握了網頁開發的「前端三劍客」：

1. **HTML (骨架)**：決定網頁有哪些內容 (文字、圖片、輸入框)。
2. **CSS (皮膚)**：決定網頁長什麼樣子 (顏色、排版、動畫)。
3. **JavaScript (靈魂)**：決定網頁能做什麼互動 (點擊按鈕、抓取資料、計算邏輯)。

從現在開始，你不再只是網頁的「瀏覽者」，而是網頁的「創造者」！